

An Explainable Approach to Process-Oriented Attacks in Industrial Control Systems using SHAP and LIME

Léa Astrid Kenmogne, Stéphane Mocanu

▶ **To cite this version:**

Léa Astrid Kenmogne, Stéphane Mocanu. An Explainable Approach to Process-Oriented Attacks in Industrial Control Systems using SHAP and LIME. IntelliSecAI 2025 - IEEE International Conference on Computational Intelligence, Security, and Artificial Intelligence, Prince Mohammad Bin Fahd University, Dec 2025, AI Khobar, Saudi Arabia. ⟨hal-05424046v2⟩

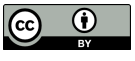
HAL Id: hal-05424046

https://hal.science/hal-05424046v2

Submitted on 19 Dec 2025 (v2), last revised 10 Jan 2026 (v3)

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

An Explainable Approach to Process-Oriented Attacks in Industrial Control Systems using SHAP and LIME

Léa Astrid KENMOGNE
Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LIG
38000 Grenoble, France
lea.kenmogne-mekemte@univ-grenoble-alpes.fr

Stéphane MOCANU
Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LIG
38000 Grenoble, France
stephane.mocanu@inria.fr

***Abstract*—Due to the increasing number of cyber threats in critical infrastructures, the security of industrial control systems (ICS) has become a major issue. These systems are at the heart of industrial installations and control critical physical processes; any alteration of which can have serious consequences for safety or production. Detection approaches have already been developed, but they have some limitations in the face of "process-oriented" attacks that act stealthily within the system by modifying the state of sensors and actuators without visibly disrupting traffic. In this paper, we propose an approach to identify these attacks based on explainable artificial intelligence (XAI) using LIME and SHAP methods to understand the model's predictions. The experimental evaluation, conducted on a simulated industrial process, demonstrates that our approach provides good detection accuracy and a comprehensive explanation of attacks, while avoiding redundant alerts typically generated by monitoring-type approaches.**

***Index Terms*—anomaly-based intrusion detection, Industrial Control Systems, XAI, SHAP, LIME.**

I. Introduction

A. Cybersecurity issues and process-oriented attacks in ICS

Due to the increasing digitalization of critical infrastructures (power grids, manufacturing industries, water treatment), the cybersecurity of industrial systems has become a major concern. Traditionally, these systems were isolated and protected by physical barriers, but today, they are interconnected with computer networks and the Internet, which exposes them and makes them vulnerable to cyber threats. Although generic threats (ransomware, denial-of-service attacks) continue to plague these systems, another, more subtle category of threat is targeting them: process-oriented attacks. These attacks directly target the physical behavior of the industrial process, manipulating sensors and actuators, while seemingly adhering to system rules, to produce unwanted effects by remaining invisible to traditional network monitoring mechanisms [1]. These attacks have been the cause of incidents such as Stuxnet [2], Maroochy Shire [3], and Industroyer [4]. The goal of these attacks is not to disrupt the network traffic, but to trigger an action that would cause the system to malfunction. Various techniques can be used to achieve this, and they are summarized in the matrix MITRE ATT&CK® [5]. It is important to have a deep understanding of the functioning of the physical process in order to protect the system and to detect these attacks; then, behavior-based detection techniques seem appropriate. This is the case, for example, of detection based on the analysis of process variable values [6], based on the regularity of traffic [7], or based on the security properties of the physical process [8], [9].

This work has been partially supported by the French National Research Agency under the France 2030 label (Superviz ANR-22-PECY-0008). The views reflected herein do not necessarily reflect the opinion of the French government.

B. Machine Learning and Explainability in Detection

Behavior-based detection techniques include several methods based on statistics, specifications, security policies, or machine learning. Statistical methods allow the normal behavior of the system to be modeled based on statistical variables. Specification-based methods establish formal rules or models of expected behavior. For policy-based methods, the principle is to compare observed actions with the rules defined in the security policy. The machine learning-based methods aim to train a model to recognize what is expected. The trained model will therefore be able to identify discrepancies between observed behavior and learned behavior. These methods are particular as they learn automatically from data, adapt to systems that are difficult to model, and can detect subtle anomalies that cause slight changes in the data. Moreover, machine learning techniques can be combined with explainability not only to detect but also to justify decisions.

The explainability of machine learning models addresses the need to make decisions that are often perceived as coming from a "black box" understandable. In the context of Industrial Control Systems (ICS), this requirement is particularly critical because the processes being monitored or controlled have a significant impact on safety, availability, and sometimes even the physical integrity of infrastructure. For human operators, understanding the origin of an alert or anomaly is essential to take appropriate corrective action. Explainability thus contributes to: i) identifying the sensors or actuators most involved in the anomaly therby locating its source more effectively; ii) analyzing the concrete impact of an anomaly on the entire

process or on a specific part of the system; iii) strengthening operators' confidence in detection tools by reducing false positives through a better understanding of cases where the model is wrong.

The rest of the paper presents state-of-the-art anomaly detection using techniques based on safety properties and machine learning. Section III describes our machine learning module. In Section IV, we present the practical case where we apply our model. The results and analyses are given in Section V, and finally, we conclude with a discussion of the work and its future perspectives.

II. State of the Art

Intrusion detection for SCADA (Supervisory Control and Data Acquisition) systems has been extensively studied [10], [11]. In [12], they emphasize the necessity of developing intrusion detection systems specifically designed for SCADA environments, given the unique communication protocols, real-time constraints, and critical safety requirements that distinguish these systems from traditional IT infrastructures. Although various methods have been proposed, we will first review detection approaches based on specifications, followed by those relying on machine learning techniques.

A. Specification patterns monitoring in ICS

As previously mentioned, we focus on detecting attacks that trigger incorrect behavior in the physical process. Correct behavior, as defined in [13], is characterized by the occurrence or non-occurrence of temporal patterns, also called specifications. From an intrusion detection perspective, the goal is to verify, during the execution of the control program, whether the physical process deviates from the correct behavior defined by the known specifications. Specification-based approaches have been studied in the literature [14]. They can be classified according to the source of the specification, the means of extracting the specification, or the method used to model it [15]. Several techniques such as model checking or runtime verification, are used in the literature [16], [17]. The specifications are expressed either in specification languages [18] or temporal logic expressions using formalisms like Linear Temporal Logic (LTL), Metric Temporal Logic (MTL), or Signal Temporal Logic (STL).

While temporal logic expressions are suitable for runtime verification-based writing, the specifications formulae for an existing system is a difficult task. For complex systems, writing correct formulae will be very difficult and even more difficult in the case of industrial systems, where it is a well-known fact that the specifications are not available or missing due to the long life of such systems and the multiplicity of service providers.

Therefore, some works [19]–[21] have studied the *specification patterns* which are commonly occurring properties. The use of "standardized" specification patterns avoids the issue of writing intricate specifications for complex systems. Some approaches [22] will attempt to identify specification patterns using a data mining technique on normal execution traces. Others will aim to deduce patterns from international standards and programming techniques [9].

In our approach, we do not use specification patterns for the detection but rather as a formalism for the explainability of the detection, i.e., we develop an AI-based detection approach that ignores the specification patterns, but we try to express the alerts' explanations in terms of specification patterns. In other words, without prior knowledge of security properties, our approach will "discover", using explainability techniques, the security properties violated by an attack.

B. Machine Learning- based IDS and Explainability in ICS

In the literature on Intrusion Detection Systems (IDS), several studies are based on machine learning [23]. In some works specific to detection in ICS, the authors train multiple models on labeled datasets containing various attack scenarios [24], [25]. When the environment is controlled, supervised techniques offer satisfactory results, but they quickly show limitations in real-world contexts, while industrial systems often lack of a variety of attacks that could be used to enrich the training database. Other works focus on neural networks or Convolutional Neural Networks (CNN) based on deep learning [26]. In [27], the proposed technique is based on a multilayer perceptron used for binary classification. The model is trained on Modbus data and assigns each instance a label indicating whether it is normal or malicious. The approach proposed in [28] leverages a CNN to capture distinctive temporal patterns in SCADA traffic and to identify the time windows in which network attacks occur.

Unsupervised machine learning techniques can also be applied, and the main advantage is that the model can be updated more easily as normal data is observed. Autoencoders are an unsupervised machine learning model widely used in the literature, specifically regarding anomaly detection, such as [29], which proposes an approach combining an LSTM (Long Short Term Memory) autoencoder with a one-class SVM (Support Vector Machine) for anomaly detection in ICS. The integration of SHAP allows the interpretation of the model's predictions, thus facilitating the analysis of anomalies.

There are several methods for explaining machine learning models. The survey [30] classifies these different methods used in cybersecurity literature in general, and the review [31] provides an in-depth analysis of XAI methods, with a particular focus on techniques used in intrusion detection systems. Among the model-agnostic methods (they can be applied to any learning model), SHAP and LIME are widely used. The main advantage of SHAP is that it can explain interactions between variables, while LIME is popular due to its simplicity. In [29], SHAP is used mainly to understand predictions, but in the work carried out in [32], explainability helps improve detection, and in [33], they develop a SHAP-based approach that allows them to identify the variables contributing to the anomaly and those offsetting the anomaly. In [34], the authors develop a model based on autoencoders and use SHAP to identify the variables and interactions between variables that caused the model to make the decision.

III. Methodology

A. Machine Learning Model

We are working on unsupervised techniques, specifically autoencoders, which are a type of neural network that learns non-linear representations by capturing complex structures in the data. They are made up of an encoder and a decoder, and the aim is to reconstruct the output data as close as possible to the original input data. This reconstruction is done with a reconstruction error that will be used to classify our instances in the decision process (Fig. 1, part Training Model).

We want to detect any behavior that deviates from normal, and to do this, we train our model to recognize what is normal exclusively in order to better identify any anomalies. The model is then trained on a dataset of exclusively benign data $X_1, X_2, \ldots, X_n$. Each X_i is made up of m features, and we use the autoencoder to reconstruct for each instance X'_i (of the test set) the reconstructed data $\hat{X}'_i$ is made up of the same number of features. The error for each instance is calculated using the formula

$$err_j = \frac{1}{m}\sum_{j=0}^{m}(X'_j - \hat{X}'_j)^2$$

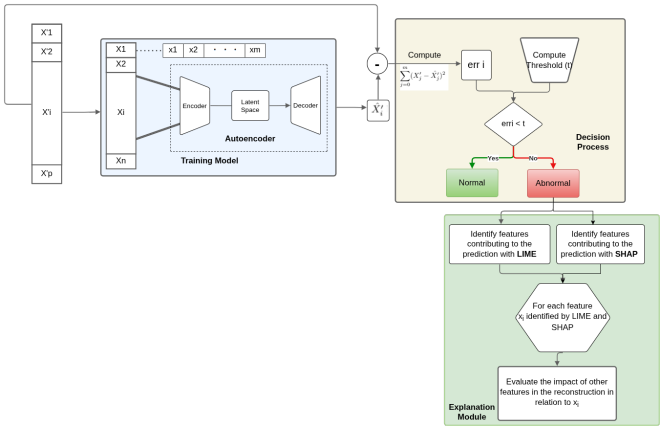


Fig. 1. XAI Detection Module

The instance classification is done based on the reconstruction error. It is compared to a defined threshold, and when the error is greater than the threshold, the instance is classified as abnormal. (Fig. 1 part Decision Process). To determine the optimal threshold, it is assumed that the majority of the data are normal and that their reconstruction scores follow a Gaussian distribution. The mean and standard deviation of the scores are then calculated, and a threshold is set based on a certain number of standard deviations above the mean.

B. Explanation Module

After detecting an anomaly, we want to know why the model classified this instance as abnormal. Understanding the reasons why the model made a prediction can be useful for the need for trust and also to optimize detection. We will analyze explainability using two different model-agnostic methods that can be applied to any learning model.

SHAP (SHapley Additive exPlanations) is a game-theory-based method [35]. It assigns each feature of an instance a weight, its SHAP value, representing its contribution to the model's prediction. SHAP values are computed by considering all feature combinations to ensure a consistent explanation. The method is both local (for individual predictions) and global (across the model), allowing identification of feature interactions. For instance X, the prediction is given by

$$g(X) = \phi_0 + \sum_{j=1}^{m}\phi_j X_j$$

where ϕ_j is the contribution of the feature j (SHAP value) for the instance X,

$$\phi_j = \sum_{S\subseteq\{1,2,\ldots,m\}\setminus\{j\}} \frac{|S|!(m-|S|-1)!}{m!}[g(S\cup\{j\}) - g(S)]$$

with S the subset of the features excluding the j-th feature.

LIME (Local Interpretable Model-agnostic Explanations) is a local explanation method that interprets a model's decisions by replacing it, for a given prediction, with a simple and interpretable (often linear) model trained locally around the instance to be explained. It slightly perturbs the input data, observes prediction variations, and adjusts a comprehensible model to extract the most influential features. It approximates the prediction around an instance X with a linear model g

$$\arg\min_{g\in G}\ L(f, g, \pi_X) + \Omega(g)$$

With f the complex model to be explained, g the interpretable model, L the loss, which defines measures how close the explanation is to the prediction of the original model; π_X the proximity measure, and $\Omega(g)$ the model complexity of g.

IV. Use case

In ICS, PLCs (Programmable Logic Controllers) execute control logic by processing sensor data and driving actuators. The interface between the physical process and the user is provided by a SCADA module, which monitors the process. For efficient, centralized, and strategic traffic monitoring, we place our detection module between the SCADA module and the PLC to detect network attacks that could compromise system security and availability. In our case study, the supervision module communicates with the PLC, sending control commands and receiving sensor status data.

A. Process Overview

The system studied is based on a hardware-in-the-loop simulation of an industrial process controlled by a real PLC. The simulation reproduces the operation of the physical system, including sensors and actuators, and interfaces with the PLC through electronic cards. The process corresponds to a chemical system described in [8] (Figure 2) and involves

two mixing sub-processes with transfers between the tanks. A weak dependency exists between the sub-processes due to product transfer, allowing identification of two distinct activities. The first activity controls T1, while the second manages Tp, T2, and the silo. The process begins with the sequential introduction of two substances into T1 via valves VP1 and VP2 until predefined levels P1 and P2 are reached, after which motor M1 ensures proper mixing. The mixed fluid is then discharged into buffer tank Tp via valve VT1, marking the transition to the second activity. Synchronization occurs during the transfer from T1 to the buffer controlled by the second activity.

In the subsequent process, the contents of Tp are directed into tank T2 by opening valve VT2. Next, a third component is transported by wagon W, which moves from point sb to sh using motion actuator M, before being added to T2. The wagon then returns to its initial position (sb) thanks to the actuator D. Then, the mixture is finalized using motor M2. Finally, the product obtained is sent to a silo, via valve VT2, where the aim is to maintain a constant level of liquid between nb and nh. In our process, we have 2 main types of components:

- **Digital sensors**.
  - Level sensors detect the level of product in the tanks (P1, P2, nb, nh).
  - Presence sensors detect the presence of the respective products in the tanks (T1vid, Tpvid, T2vid, Svid).
  - Position sensors detect the physical position of the wagon (sh, sb).
- **Digital actuators**.
  - Valve actuators: mechanically operate a valve according to instructions received from the control system (VP1, VP2, VT1, VTp, VT2, VS).
  - Electromechanical actuators (motors), which mix the products in the tanks (M1, M2).
  - Motion actuators, which start the movement of the wagon (M, D).

The control program written in Sequential Functional Chart (SFC) language corresponding to the described behavior is represented in Figure 3. The two activities are identified.

B. Data Description and Operational Workflow

Data is collected at two levels: (i) from the HMI-PLC channel, in the form of Modbus network traces (pcap files), and (ii) from the process simulation, which generates a timestamped log of the values recorded by sensors and actuators throughout the simulation [8]. We filter the traffic (pcap files) so as to keep only the read, write request, and answers of the Modbus protocol, and then using a Python script, we extract the digital sensors and actuators data. This data pre-processing is performed on two distinct files: one containing only normal traffic, and another in which anomalies have been introduced. The resulting files constitute the datasets.

In Tables I and II, we present an extract from the normal data that allows us to describe the complete operational flow of our process. All 20 variables could be represented in the same

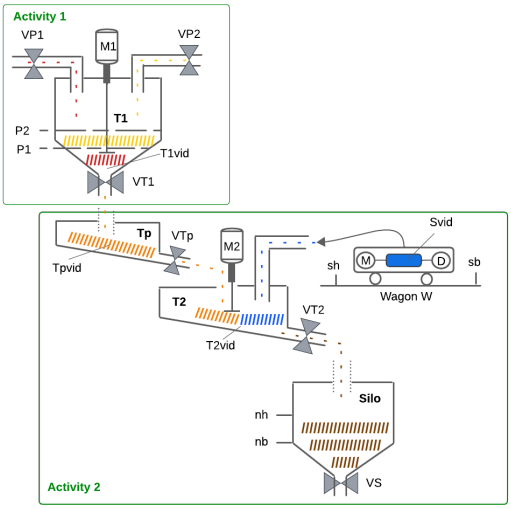


Fig. 2. Process

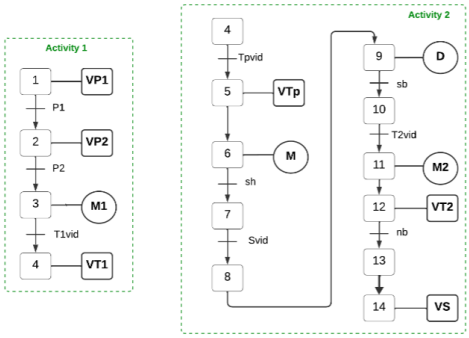


Fig. 3. Control program divided into activities

table, but since both activities can occur simultaneously, we present the actions of Activity 1 first and then those of Activity 2. This enables us to focus only on the variables involved in the activity. For the sake of representation, we do not include all the consecutive lines of the dataset but only those that bring a change. For example, in Table I, lines 2-9 are identical to line 1.

TABLE I
DATA EXTRACT FOR THE PROGRESS OF ACTIVITY 1

Index	P1	P2	T1vid	VP1	VP2	M1	VT1
1	0	0	1	1	0	0	0
.	.	.	.	.	.	.	.
10	1	0	0	0	1	0	0
.	.	.	.	.	.	.	.
20	1	1	0	0	0	1	0
.	.	.	.	.	.	.	.
67	1	1	0	0	0	0	1

When activity 1 is launched, tank T1 is still empty, with no product. We start by putting the first product, and for this, we open the valve VP1 (index 1-9) (Table I). When we reach level P1, we close VP1 and we open VP2 to fill the tank with

TABLE II
DATA EXTRACT FOR THE PROGRESS OF ACTIVITY 2

Index	Tpvid	VTp	T2vid	sb	Svid	M	sh	D	M2	VT2	nb	nh	VS
69	0	0	1	1	0	0	0	0	0	0	0	0	0
.	.	.	.	.	.	.	.	.	.	.	.	.	.
71	0	1	1	1	0	0	0	0	0	0	0	0	0
.	.	.	.	.	.	.	.	.	.	.	.	.	.
75	1	1	0	1	0	1	0	0	0	0	0	0	0
.	.	.	.	.	.	.	.	.	.	.	.	.	.
87	1	0	0	0	0	0	1	0	0	0	0	0	0
88	1	0	0	0	1	0	1	1	0	0	0	0	0
.	.	.	.	.	.	.	.	.	.	.	.	.	.
100	1	0	0	1	1	0	0	0	1	0	0	0	0
.	.	.	.	.	.	.	.	.	.	.	.	.	.
157	0	0	0	1	0	0	0	0	0	1	0	0	0
.	.	.	.	.	.	.	.	.	.	.	.	.	.
159	0	0	1	1	0	0	0	0	0	0	0	0	0
.	.	.	.	.	.	.	.	.	.	.	.	.	.
161	0	0	0	1	0	0	0	0	0	0	1	0	0
.	.	.	.	.	.	.	.	.	.	.	.	.	.
180	1	0	0	0	1	0	0	1	0	0	0	0	1

product 2 up to P2 (index 10-19). When the two products have been introduced into the tank, the mixing can begin thanks to the motor, which is engaged (index 20). The motor runs for a certain time (index 21-67), and the mixture can be transferred to the temporary tank Tp through VT1 (index 67), which marks the beginning of activity 2.

The product is in tank Tp and therefore Tp is no longer empty (index 69) (Table II). Valve VTp is opened to transfer the mixture into tank T2 (index 71-74). When all the mixture has been discharged, VTp is closed again, Tp is empty again, and the movement of wagon W is started via actuator M (index 75). The movement of the wagon continues until it reaches position sh (sh=1, index 87) and the product is transferred from the wagon to tank T2. Svid changes to 1 as it is empty again, and the return of the wagon is started using drive D (index 88). Only when the trolley reaches position sb, the rotation of motor M2 for the mixture can start (index 100). The engine takes some time to perform the mixing (index 100-156), and when the mixing is finished, the VT2 valve is opened to transfer it into the silo (index 157). T2 is completely emptied (index 159), and we make sure to reach at least the level nb in the silo before opening VS (index 161). Finally, the VS valve is opened to empty the silo (index 180). The cycle starts again. Note that while the first cycle is continuing in activity 2, another cycle has been launched in activity 1, and we can see, for example, that the wagon which was empty at index 100 (Svid=1) is no longer empty at index 161 (Svid=0).

C. Attack Scenarios and Security Design Patterns

The normal behavior of our process is deterministic and structured. It is based on a sequence of actions, whose order is important. On this basis, we can identify any legitimate behavior, and we can describe any behavior that deviates from the expected sequence as potentially abnormal. It is therefore difficult to have an exhaustive list of possible anomalies in our system, and this makes our process suitable for behavior-based anomaly detection. Nevertheless, in Table III, we present the attacks that were deliberately included during the simulation and the security patterns that were used for detection in [8].

V. RESULTS AND DISCUSSION

A. Detection level

The effect of an attack will be visible on several consecutive records of our dataset. For example, in the case of Activity 1, there are 5 types of attack (A1-A5), and the effects of these attacks are spread over a total of 38 lines. In the case of Activity 2, 3 types of attack have been injected, and these are repeated over 37 lines. The normal and test datasets used in this paper, together with the code used for the explainability, are available at: https://github.com/datasetXAI/xai.

For Activity 1, our model correctly detects the 38 abnormal lines, but we have 2 false positives. For activity 2, we also have 2 false positives. The cases of false positives correspond, as in activity 1, to two normal instances that the model classifies as abnormal, while they were not present in the training data set. These false positives, due to the incompleteness of the normal dataset, were also mentioned in [8].

B. Explainability analysis

In order to better understand the factors that led to the detection of anomalies by our autoencoder, we used two explainability methods: LIME and SHAP. For each instance, we first evaluate the impact of the variables on the reconstruction error using each of the two methods. This gives us a fairly global overview of the variables that pushed the instance towards prediction. The idea is to dig deeper into explainability at a second level, by trying to understand why each of the variables that pushed towards the prediction is important, or what correlations this variable had with the others, so that our explainability algorithms consider it to be preponderant in our prediction. This second-level study will be carried out on the variables identified by SHAP and LIME, enabling us to validate the robustness of the explanations when the two approaches converge. In addition, any discrepancies that may arise between the two approaches will allow us to detect other abnormal behaviors in certain cases. We detect all attacks and, for each one, all abnormal lines. However, due to space constraints, we will explain only a few of them.

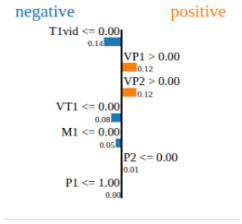
Attack A1: Opening VP1 when P1 is reached

Attack A1 effects are detected in 7 consecutive records of the abnormal dataset (index 287-293). In order to better understand the model's predictions, we study each line individually before drawing overall conclusions. Figures 4a and 4b show the variable contributions to prediction for instance 287, and we can see that for LIME and SHAP the common variables that contributed most are VP1 and VP2. With SHAP (Fig. 4b), we note the contribution of variable P1 to the reconstruction error (which does not occur in LIME), and this seems rather correct, allowing us to identify a security property that has been violated ***never(P1 and VP1)***. At this level, we have no clear way of knowing in which order the actions took

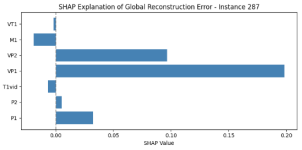
TABLE III
ATTACKS

N°	Attack Description	Security Pattern used in [8]	Pattern Code
A1	**Opening VP1 when P1 is reached**	*Never(VP1_Open)After(P1_reached)*	**[P1]**
A2	**Starting M1 with VP1 open**	*Always(VP1_closed)Before(M1_run)*	**[P2]**
A3	**Opening VP2 when M1 runs**	*Always(VP2_closed)Before(M1_run)*	**[P3]**
A4	**Opening VT1 when M1 runs**	*Always(VT1_closed)Before(M1_run)*	**[P4]**
A5	**Starting M1 with T1 empty**	*Always(! Tp_empty)Before(M1_run)*	**[P5]**
A6	**Start M2without sb**	*Always(sb)Before(M2_run)*	**[P6]**
A7	**Opening VT2 when M2 runs**	*Always(VT2_closed)Before(M2_run)*	**[P7]**
A8	**Opening VTp when Tp is empty**	*Never(VTp)After(Tp_empty)*	**[P8]**

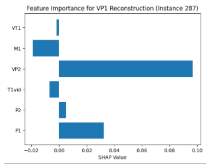
place (VP1 opened before P1 was reached, or P1 was reached before VP1 opened) to put the security property in the form of a pattern as in Table III. To know the order, we have to look at the dataset, but the order does not change the fact that it remains an anomaly. We only know that LIME and SHAP indicate a violation of the safety property involving VP1 and VP2 (***never(VP1 and VP2)***), and that SHAP additionally reveals another violated safety property. For the features VP1 and VP2, it is therefore important to continue the investigation to understand what has impacted the different interactions in the reconstruction of VP1 and VP2. We do this with the help of SHAP and can plot the results shown in Figures 4c and 4d. The graphs have been plotted for VP1 and VP2, but in reality, it is the contribution of VP2 that may seem surprising. In Figure 4d, we can see that VP1 has made a major contribution to the reconstruction of VP2. Revisiting the dataset in depth (an extract of which is shown in Table IV), we notice the security property that has been violated: *"VP1 and VP2 shall not be open simultaneously"*.



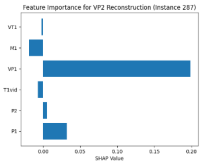
(a) LIME



(b) SHAP



(c) VP1



(d) VP2

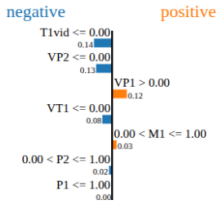
Fig. 4. A1 Explainability (instance 287)

A similar analysis is performed on lines 288-290, and the graphs are the same as line 287 since both security patterns are violated on the first 4 lines of the 7 abnormal ones. By performing a similar study on index line 291, the results obtained are different, and we present them in Figure 5.

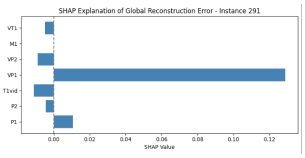
TABLE IV
DATASET RECORDS FOR A1.

Index	P1	P2	T1vid	VP1	VP2	M1	VT1
287	1	0	0	1	1	0	0
288	1	0	0	1	1	0	0
289	1	0	0	1	1	0	0
290	1	1	0	1	1	0	0
291	1	1	0	1	0	1	0
292	1	1	0	1	0	1	0
293	1	1	0	1	0	1	0

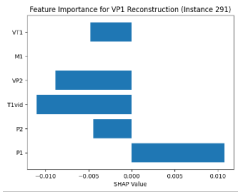
The graphs returned by LIME and SHAP (Figures 5a and 5b) for the instance 291 show that both algorithms give the variable VP1 as a major contributor to the error. SHAP returns, in addition to VP1, the variable P1, which is normal, and the justification is the same as 287. LIME returns VP1 but also M1, and this suggests that there is an anomaly involving these two variables or a security property violated: ***never(VP1 and M1)***. By moving on to the second level of investigation for variables VP1 and M1 with SHAP, Figure 5c shows that P1 makes a major contribution to the reconstruction of VP1 (that corresponds to ***never(P1 and VP1)***). The Figure 5d highlights a strong contribution of VP1 to the reconstruction of M1 which allow us to identify the violation of the security property *"M1 and VP1 shall not be activated simultaneously"*.



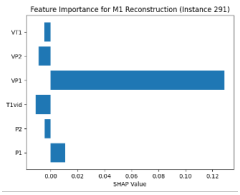
(a) LIME



(b) SHAP



(c) VP1



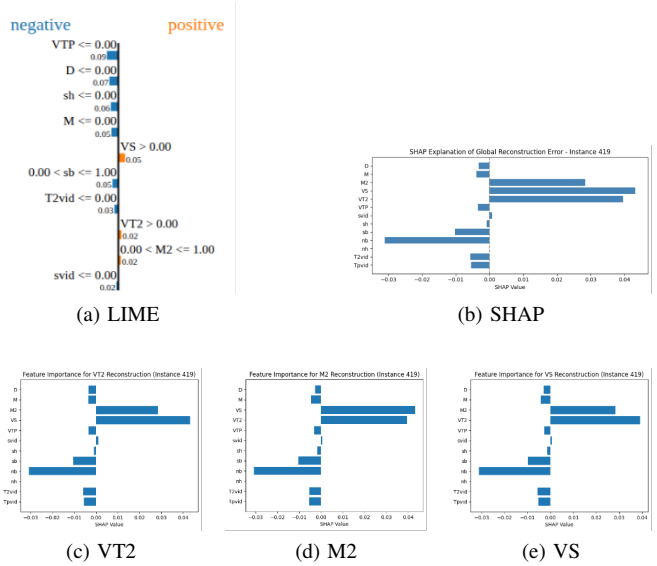
(d) M1

Fig. 5. A1 Explainability (instance 291)

The line-by-line explainability study on the A1 attack identified two other security properties that were violated. For the first lines (index 287-290), LIME and SHAP identified the other violated security property, and SHAP alone justified the model's prediction regarding the initial pattern. For the last 3 lines (index 291-293), we were able to identify the other violated security property primarily through LIME.

Attack A7: Opening VT2 when M2 runs

This attack attempts to empty the T2 tank while mixing is still in progress. The global analysis of instance 547 reveals, both by LIME and by SHAP, that the variables VS, VT2, and M2 contributed strongly to the model's decision (Fig. 6a and 6b). In addition, SHAP analysis of the reconstruction of each of these 3 variables reveals that all 3 interact: M2 and VS contribute to the reconstruction of VT2 (Fig. 6c), VS and VT2 contribute to the reconstruction of M2 (Fig. 6d), and M2 and VT2 contribute to the reconstruction of VS (Fig. 6e). The least obvious contribution to this anomaly is that of VS and, looking deeper into the dataset, we realize that there is another safety property that has been violated: ***never(VT2 and VS)***. In normal operation, while the product is being transferred from VT2 to the silo, the VS valve must be closed to ensure that the desired nb level is maintained before emptying the silo, so, we should not *"Open VT2 and VS simultaneously"*.



(a) LIME (b) SHAP

(c) VT2 (d) M2 (e) VS

Fig. 6. A7 Explainability

C. Discussion

Anomaly explanation is achieved by combining two methods (LIME and SHAP), thus increasing confidence when both methods converge on the same explanatory features. When one of the methods gives important variables that the other does not, this can reveal more subtle aspects of the abnormal behavior or alternative patterns that could otherwise go unnoticed. In the case of attack A1, a total of 3 security properties were violated. SHAP exclusively identified one property (instance 287), LIME also exclusively identified one property (instance 291), and both methods identified the violation of the third security property. In the case of attack A7, the two methods converge, and we use SHAP to study the interactions between the various variables of the anomaly.

The explanation is performed at two levels: the first, using SHAP and LIME, identifies the most significant variables; the second, using SHAP, evaluates the contribution of other variables to the reconstruction of the important ones. This enables precise detection of variable interactions that cause the violation of security properties.

High detection accuracy. This result is all the more remarkable given that our approach relies on very limited information: the state of the sensors and actuators, without detailed knowledge of the system's internal functioning. The model, trained solely on this data, nevertheless manages to capture the logic of the system's normal behavior. This allows it to distinguish abnormal deviations, identifying anomalies even for weak signals.

Deep and comprehensive detection. Attacks are detected, as well as their impact and consequences on the system. The model identifies not only when the attack occurs but also the persistence of its effects over time.

One of the limitations of our approach is that it does not take into account temporal aspects and continuous variables. For example, the requirement for a motor to run for 50 seconds is not modeled, and variables such as temperature and pressure, which evolve continuously, are not yet integrated.

Another limitation of our approach is that it is restricted to sequential systems and still requires some (limited) knowledge of the PLC program (namely, the activities). Control devices such as PID (Proportional Integral Derivative) and similar cannot be integrated into the current framework.

With respect to the runtime-monitoring approach, we notice that we do not need to mine for specifications or extract them from standards and text requirements, and we find all security properties that are violated by an attack. On the other hand, our approach is unable to specify the order of the events. We will rather provide explanations on the form *never(VT1 and P1)* rather than *never(VT1) after (P1)*.

VI. Conclusion and Future Work

In this work, we proposed an approach for detecting and explaining process-oriented attacks in industrial control systems. Using only sensor and actuator values, our model successfully learns the system's normal behavior and accurately detects anomalies, even when they are infrequent or poorly reported. We developed a combined SHAP and LIME explainability method providing a detailed description of anomalous data.

In the future, we will further test the reconstruction of security properties, particularly to avoid false positives. We aim to extend the approach to larger-scale systems closer to real-world industrial systems. Finally, we will extend the method to integrate temporal and continuous variables in order to make our approach applicable to a wider variety systems.

REFERENCES

[1] F. Khorrami, P. Krishnamurthy, and R. Karri, "Cybersecurity for control systems: A process-aware perspective," *IEEE Design & Test*, vol. 33, no. 5, pp. 75–83, 2016.

[2] N. Falliere, L. O. Murchu, E. Chien, and A. Trombetta, "W32. stuxnet dossier," *White paper, Symantec Corp Security Resp.*, vol. 5, Feb. 2011.

[3] MITRE, "Malicious control system cybersecurity attack case study - maroochy water services, australia," *MITRE*, 2008, technical Report.

[4] S. McLaughlin, C. Konstantinou, X. Wang, L. Davi, A.-R. Sadeghi, M. Maniatakos, and R. Karri, "The cybersecurity landscape in ics," *Proceedings of the IEEE*, vol. 104, no. 5, pp. 1039–1057, 2016.

[5] MITRE, "Att&ck® for industrial control systems," 2021, [Online] https://attack.mitre.org/matrices/ics/, last accessed June 2025.

[6] A. Carcano, I. N. Fovino, M. Masera, and A. Trombetta, "State-based network intrusion detection systems for SCADA protocols: A proof of concept," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 6027 LNCS, pp. 138–150, 2010.

[7] R. R. R. Barbosa, R. Sadre, and A. Pras, "Exploiting Traffic Periodicity in Industrial Control Networks," *International journal of critical infrastructure protection*, vol. 13, pp. 52–62, 2016.

[8] O. Koucham, S. Mocanu, G. Hiet, J.-M. Thiriet, and F. Majorczyk, "Detecting Process-Aware Attacks in Sequential Control Systems," in *NordSec 2016 - 21st Nordic Conference on Secure IT Systems (NordSec 2016)*, Oulu, Finland, Nov. 2016, pp. p.20–36. [Online]. Available: https://inria.hal.science/hal-01361081

[9] E. Hotellier, F. Sicard, J. Francq, and S. Mocanu, "Standard Specification-based Intrusion Detection for Hierarchical Industrial Control Systems," *Information Sciences*, 2024, article 120102.

[10] S. V. B. Rakas, M. D. Stojanović, and J. D. Marković-Petrović, "A review of research work on network-based scada intrusion detection systems," *IEEE Access*, vol. 8, pp. 93 083–93 108, 2020.

[11] I. Garitano, R. Uribeetxeberria, and U. Zurutuza, "A review of scada anomaly detection systems," in *Soft Computing Models in Industrial and Environmental Applications, 6th International Conference SOCO 2011*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 357–366.

[12] B. Zhu and S. Sastry, "Scada-specific intrusion detection/prevention systems: a survey and taxonomy," in *Proceedings of the 1st workshop on secure control systems (SCS)*, vol. 11, 2010, p. 7.

[13] O. Maler, "Some thoughts on runtime verification," in *Runtime Verification - 16th International Conference, RV 2016, Madrid, Spain, September 23-30, 2016, Proceedings*, 2016, pp. 3–14.

[14] C. Ko, M. Ruschitzka, and K. Levitt, "Execution monitoring of security-critical programs in distributed systems: a specification-based approach," in *Proceedings. 1997 IEEE Symposium on Security and Privacy (Cat. No.97CB36097)*, 1997, pp. 175–187.

[15] L. Nweke, "A survey of specification-based intrusion detection techniques for cyber-physical systems," *International Journal of Advanced Computer Science and Applications*, vol. 12, pp. 37–45, 05 2021.

[16] T. Jacob, M. Raman, and S. Singh, *Intrusion Detection in Zero Knowledge System Using Model Checking Approach*, 01 2013, pp. 453–465.

[17] A. Dokhanchi, B. Hoxha, and G. Fainekos, "On-line monitoring for temporal logic robustness," *CoRR*, vol. abs/1408.0045, 2014. [Online]. Available: http://arxiv.org/abs/1408.0045

[18] E. Bartocci, Y. Falcone, A. Francalanza, and G. Reger, "Introduction to Runtime Verification," in *Lectures on Runtime Verification. Introductory and Advanced Topics*, ser. Lecture Notes in Computer Science. Springer, Feb. 2018, vol. 10457, pp. 1–33. [Online]. Available: https://inria.hal.science/hal-01762297

[19] M. B. Dwyer, G. S. Avrunin, and J. C. Corbett, "Patterns in property specifications for finite-state verification," in *Proceedings of the 21st international conference on Software engineering*, 1999, pp. 411–420.

[20] S. Konrad and B. Cheng, "Real-time specification patterns," in *Proceedings of the 27th international conference on Software engineering*. New York, NY, USA: ACM, 2005, pp. 372–381.

[21] O. Maler and D. Ničković, "Monitoring Temporal Properties of Continuous Signals," in *International Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems*. Springer, 2004, pp. 152–166.

[22] O. Koucham, S. Mocanu, G. Hiet, J.-M. Thiriet, and F. Majorczyk, "Efficient Mining of Temporal Safety Properties for Intrusion Detection in Industrial Control Systems," in *SAFEPROCESS*, 2018, pp. 1–8.

[23] H. Liu and B. Lang, "Machine learning and deep learning methods for intrusion detection systems: A survey," *Applied Sciences*, vol. 9, p. 4396, 10 2019.

[24] A. Keliris, H. Salehghaffari, B. Cairl, P. Krishnamurthy, M. Maniatakos, and F. Khorrami, "Machine learning-based defense against process-aware attacks on industrial control systems," in *2016 IEEE International Test Conference (ITC)*, 2016, pp. 1–10.

[25] A. Robles, N. Moradpoor, J. McWhinnie, and G. Russell, "A supervised energy monitoring-based machine learning approach for anomaly detection in a clean water supply system," in *Proceedings of the Ieee international conference on cyber security and protection of digital services*, 06 2018, pp. 1–8.

[26] Y. Lai, J. Zhang, and Z. Liu, "Industrial anomaly detection and attack classification method based on convolutional neural network," *Security and Communication Networks*, vol. 2019, pp. 1–11, 2019.

[27] A. Hijazi and J.-M. Flaus, "A deep learning approach for intrusion detection system in industry network," in *The first international conference on Big Data and Cybersecurity intelligence*, 02 2019, pp. 55–62.

[28] H. Yang, L. Cheng, and M. C. Chuah, "Deep-learning-based network intrusion detection for scada systems," in *2019 IEEE Conference on Communications and Network Security (CNS)*, 2019, pp. 1–7.

[29] D. T. Ha, N. X. Hoang, N. V. Hoang, N. H. Du, T. T. Huong, and K. P. Tran, "Explainable anomaly detection for industrial control system cybersecurity," *IFAC-PapersOnLine*, vol. 55, no. 10, pp. 1183–1188, 2022, 10th IFAC Conference on Manufacturing Modelling, Management and Control MIM 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896322018572

[30] F. Charmet, H. C. Tanuwidjaja, S. Ayoubi, P.-F. Gimenez, Y. Han, H. Jmila, G. Blanc, T. Takahashi, and Z. Zhang, "Explainable artificial intelligence for cybersecurity: a literature survey," *Annals of Telecommunications - annales des télécommunications*, vol. 77, no. 11-12, pp. 789–812, Dec. 2022. [Online]. Available: https://hal.science/hal-03965590

[31] S. Neupane, J. Ables, W. Anderson, S. Mittal, S. Rahimi, I. Banicescu, and M. Seale, "Explainable intrusion detection systems (x-ids): A survey of current methods, challenges, and opportunities," *IEEE Access*, vol. 10, pp. 112 392–112 415, 01 2022.

[32] K. Roshan and A. Zafar, "Utilizing xai technique to improve autoencoder based model for computer network anomaly detection with shapley additive explanation(shap)," *International journal of Computer Networks & Communications*, vol. 13, no. 6, p. 109–128, Sep. 2021. [Online]. Available: http://dx.doi.org/10.5121/ijcnc.2021.13607

[33] L. Antwarg, R. M. Miller, B. Shapira, and L. Rokach, "Explaining anomalies detected by autoencoders using shapley additive explanations," *Expert Systems with Applications*, vol. 186, p. 115736, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417421011155

[34] L. A. Kenmogne and S. Mocanu, "Explainable ai for process-aware attack detection in industrial control systems," *IEEE 10th International Conference on Network Softwarization (NetSoft)*, pp. 363–368, 2024.

[35] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 4768–4777.